

Monitorización y control de PCs mediante ESP32 y panel LCD (TFG).

Alumno: Iván Rodríguez Moraleda.

Fecha: 12/06/2024.

Curso: 2º STI (Quintanar de la Orden).

Índice:

1. MEMORIA..... 3

1.2.2. 1. INTRODUCIÓN..... 3

1.1.1. Especificaciones del cliente. 3

1.1.2. Justificación de la solución adoptada. 4

1.3. Objetivos. 6

1.4. Estudio de viabilidad..... 6

1.2. Desarrollo. 8

1.2.1. Esquema de conexionado. 8

1.2.3. Componentes Hardware 12

1.2.3. Desarrollo software..... 17

1.3. Validación y conclusiones. 18

1.3.2. Presupuesto. 19

3. Planos..... 20

3.1. Montaje eléctrico. 20

1.3.3. 4. Anexos. 21

1.3.4. 4.1. Código fuente. 21

1.3.5. 4.2.Manuales..... 25

1.3.6. 4.2.1.Manual de uso..... 25

1.3.7. 4.2.2.Manual de mantenimiento. 27

1.3.8. 5.Bibliografía. 29

1. MEMORIA

1.1. INTRODUCIÓN

1.1.1.Especificaciones del cliente.

Planteamiento del Problema

PictureREC es una empresa especializada en diseño gráfico y animación, donde se realizan proyectos que requieren un alto rendimiento gráfico, como la creación de videos y gráficos en 3D. Con el fin de llevar a cabo estas tareas, la empresa cuenta con dos estaciones de trabajo de alta gama ubicadas en una sala de servidores dedicada. Estas estaciones de trabajo desempeñan un papel crítico en el proceso de renderizado, siendo esenciales para el flujo de trabajo creativo.

Dado que estas estaciones de trabajo necesitan ser encendidas y apagadas según las demandas del equipo creativo, el acceso a ellas es crucial. Sin embargo, debido a las estrictas políticas de seguridad y a la ubicación de la sala de servidores en un área de acceso restringido, el equipo de TI enfrenta el desafío de controlar las computadoras sin la necesidad de entrar físicamente a la sala.

El equipo de TI de PictureREC evita entrar a la sala de servidores debido a razones de seguridad, para proteger los datos y equipos sensibles. Además, evitan el riesgo de dañar accidentalmente los equipos costosos que están dentro. Limitar el acceso también ahorra tiempo y recursos, ya que pueden controlar los equipos de forma remota, sin la necesidad de desplazarse físicamente. Esto ayuda a mantener las operaciones de manera eficiente y segura.

A lo largo de este proyecto, se explorarán los desafíos técnicos, las soluciones encontradas y el proceso de desarrollo del sistema de control remoto. Se espera que esta solución mejore la eficiencia y la seguridad en el manejo de las estaciones de trabajo de PictureREC, facilitando el flujo de trabajo creativo y optimizando sus operaciones.

Requisitos

- Encendido y Apagado Remoto:

Capacidad para encender y apagar los ordenadores de forma remota a través de una conexión en la misma red.

- Monitorización de Estado:

Funcionalidad para monitorear el estado de los ordenadores, conocer si está encendido o apagado.

- Interfaz Sencilla:

Desarrollo de una interfaz de usuario intuitiva y fácil de usar que permita a los usuarios controlar remotamente las estaciones de trabajo con facilidad. Esta interfaz se deberá ver en un pequeño display.

- Mando Empotrado en la Pared:

Diseño y creación de un dispositivo de control físico que estará empotrado en la pared de una de las salas de la oficina. Este dispositivo actuará como un "mando a distancia" para encender y apagar las estaciones de trabajo de manera local.

1.1.2.Justificación de la solución adoptada.

Diferentes microcontroladores del mercado:

- Arduino Uno o Arduino Mega: Estas placas Arduino son ampliamente utilizadas y cuentan con una comunidad sólida de usuarios y una gran cantidad de recursos disponibles. Puedes utilizarlas junto con un módulo Ethernet o Wi-Fi para la conectividad remota.
- Raspberry Pi: Si prefieres una solución más potente y versátil, podrías utilizar una Raspberry Pi. Esta pequeña computadora es capaz de ejecutar un sistema operativo completo como Linux y ofrece opciones de conectividad como Wi-Fi y Ethernet.
- NodeMCU: Similar a la ESP32, NodeMCU es una plataforma de desarrollo basada en el microcontrolador ESP8266. Ofrece conectividad Wi-Fi y es compatible con el entorno de desarrollo de Arduino, lo que facilita la programación.
- Particle Photon o Particle Electron: Estas placas son excelentes opciones si buscas una solución IoT listo para usar. Ofrecen conectividad Wi-Fi o celular (en el caso de Electron) y cuentan con una plataforma en la nube para la gestión remota.
- BeagleBone Black: Esta placa es otra alternativa similar a la Raspberry Pi. Tiene capacidades de conectividad y potencia de procesamiento.
- ESP32: es una microcontroladora de bajo costo y alto rendimiento que ofrece conectividad Wi-Fi y Bluetooth integrada, ideal para proyectos IoT y aplicaciones embebidas donde se requiere potencia de procesamiento y comunicación inalámbrica.

Si bien otras opciones como Raspberry Pi y Particle Photon también tienen sus ventajas, opté por ESP32:

- Conectividad Integrada: El ESP32 ofrece conectividad Wi-Fi y Bluetooth integrada, lo que lo hace ideal para aplicaciones IoT que requieren comunicación inalámbrica.
- Potencia de Procesamiento: Con un procesador de doble núcleo y una velocidad de reloj de hasta 240 MHz, el ESP32 ofrece una potencia de procesamiento significativamente mayor en comparación con placas como Arduino Uno o Raspberry Pi Zero. Esto permite ejecutar aplicaciones más complejas y procesar grandes cantidades de datos de manera eficiente.
- Bajo Consumo de Energía: A pesar de su potencia de procesamiento, el ESP32 está diseñado para ser altamente eficiente en términos de consumo de energía.
- Compatibilidad con Arduino: El ESP32 es compatible con el entorno de desarrollo de Arduino.
- Amplia Comunidad y Soporte: Gran cantidad de recursos, tutoriales y proyectos disponibles en línea, así como una comunidad activa de desarrolladores dispuestos a ayudar y compartir conocimientos.
- Costo Inicial**:** El ESP32 suele tener un costo inicial más bajo en comparación con placas como Raspberry Pi o Particle Photon.

Conectaré el ESP 32 a un LCD para visualizar la interfaz y poder controlarla gracias a tres pulsadores.

Diferentes tecnologías:

- Wake-on-LAN (WoL): WoL es un protocolo que permite encender dispositivos de forma remota a través de la red local.
- WEBSERVER: Un servidor web permite controlar los dispositivos de forma remota a través de una interfaz web.
- SSH (Secure Shell): SSH es un protocolo que permite la comunicación segura con dispositivos remotos a través de la línea de comandos.
- MQTT (Message Queuing Telemetry Transport): MQTT es un protocolo de mensajería ligero diseñado para dispositivos IoT.

Opté por el servidor web ya que proporciona varias ventajas:

- Mayor facilidad de configuración a diferencia de MQTT O SSH.
- No requiere Aplicaciones Adicionales para visualizar los mensajes de manera gráfica, necesitas una aplicación extra.
- Además, a diferencia de WoL, puede apagar remotamente un dispositivo.
- Puede proporcionar un punto de acceso centralizado para la gestión y control de los dispositivos, lo que simplifica el proceso. En lugar de tener que configurar y mantener múltiples conexiones individuales utilizando protocolos como MQTT o SSH.

¿Cómo se enlazaría con los demás componentes? En resumen, el ESP32 y su display LCD proporcionan la interfaz física para que los usuarios remotos interactúen con el sistema de control y monitorización. Sin embargo, el servidor Flask, ejecutándose en una ubicación central y accesible, es el responsable de recibir las solicitudes del ESP32, procesarlas y ejecutar los comandos necesarios para controlar los servidores PC 1 y PC 2, incluyendo el apagado/encendido cuando sea solicitado.

1.1.3.Objetivos.

- Investigación y Selección del Hardware: Investigar y seleccionar el hardware necesario para el proyecto, incluyendo microcontroladores, sensores, actuadores y dispositivos de red.
- Diseño del Sistema: Diseñar la arquitectura del sistema, incluyendo la conexión de hardware, la comunicación entre dispositivos y la interfaz de usuario.
- Configuración de IPs y Conexión de Red: Configurar las direcciones IP para los dispositivos y asegurar una conexión estable a la red local.
- Desarrollo del Código: Escribir el código necesario para cada dispositivo, incluyendo el código para hacer ping a los equipos, encender y apagar remotamente, y configurar las IPs. Utilizar lenguajes de programación como C o Python.
- Pruebas y Depuración: Realizar pruebas exhaustivas del sistema para verificar su funcionamiento correcto. Depurar cualquier error o problema que surja durante las pruebas.
- Implementación del Hardware: Conectar y montar físicamente el hardware en una de las salas cercana a los servidores de PictureREC, asegurándose de que esté correctamente instalado y funcionando.
- Integración con Interfaz de Usuario: Desarrollar una interfaz de usuario intuitiva y fácil de usar.

1.1.4.Estudio de viabilidad.

<u>Técnica:</u>

- Evaluamos si es posible desarrollar nuestros productos y servicios de manera eficiente desde el punto de vista tecnológico. Esto incluye la seguridad, el control, las características, funcionalidades y propiedades físicas del proyecto, así como los métodos de ejecución.
- Monitorización del Estado de Ordenadores: Nuestra empresa está altamente capacitada en la implementación de sistemas para monitorizar el estado de ordenadores en tiempo real. Utilizamos ESP32 para recopilar datos sobre el rendimiento y el estado de los equipos, enviando esta información a un LCD para su supervisión.
- Control Remoto de Ordenadores: Contamos con la tecnología y el personal especializado para instalar sistemas de control remoto que permiten encender, apagar y reiniciar ordenadores de manera remota a través de una red Wi-Fi, utilizando el ESP32.

- Programación de Sistemas: Tenemos un equipo especializado en la programación y puesta en marcha de automatismos electrónicos con ESP32, enfocados en el control y monitorización de ordenadores. Esto incluye la integración de sensores y actuadores para un control preciso y fiable. Nuestro equipo es capaz de desarrollar soluciones personalizadas que se adaptan a las necesidades específicas de cada cliente, asegurando la interoperabilidad con sistemas existentes.
- Sistemas de Comunicación Inalámbrica: En proyectos que requieren comunicación entre diferentes edificios o áreas difíciles de cablear, nuestra empresa está cualificada para montar sistemas de comunicación inalámbrica. Utilizamos tecnología fiable para asegurar conexiones estables y de alta velocidad, lo que es crucial para la monitorización y control remoto de ordenadores.

Económica:

Nuestra empresa cuenta con un sólido fondo económico que permite realizar instalaciones de cualquier tamaño. Evaluamos los aspectos financieros del proyecto, cuantificando ingresos y egresos en el presupuesto.

Laboral:

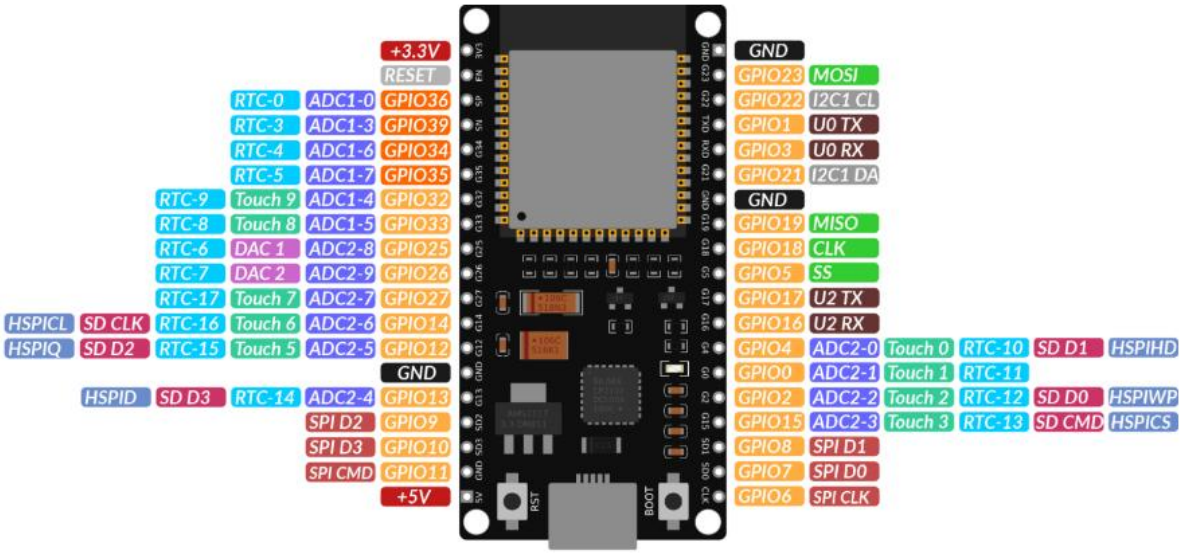
Tenemos un amplio equipo de trabajadores especializados en diversas áreas, todos con formación específica y experiencia en control remoto y monitorización de ordenadores con ESP32. Nuestra política incluye capacitación continua para asegurar la alta calidad del trabajo. Además, contamos con un equipo de soporte técnico disponible 24/7 para atender cualquier eventualidad, garantizando la continuidad operativa de nuestros servicios.

Legal:

Cumplimos con todos los permisos, licencias y certificaciones de calidad requeridos por la normativa vigente. Nos aseguramos de que el proyecto respete la ley y contribuya a la conservación del medio ambiente. Esto incluye cumplir con las normativas de protección de datos, seguridad de la información y regulaciones específicas del sector tecnológico. Además, trabajamos en estrecha colaboración con entidades regulatorias para garantizar que nuestros productos y servicios cumplen con los más altos estándares de calidad y sostenibilidad.

1.2.Desarrollo.

1.2.1.Esquema de conexionado.



1. **Puertos Digitales de Entrada/Salida (todos soportan PWM)**: Son pines que pueden usarse para conectar dispositivos y enviar o recibir señales digitales. Además, pueden generar señales analógicas usando la técnica PWM.
2. **Puertos de Entrada Digital**: Son pines que pueden leer señales digitales, útiles para detectar si un dispositivo está encendido o apagado.
3. **Entradas Analógicas de 12 bits, 0 a 3.3V**: Son pines que pueden medir voltajes analógicos, útiles para leer sensores como potenciómetros o sensores de temperatura.
4. **Salidas Analógicas de 8 bits, 0 a 3.3V**: Son pines que pueden generar voltajes analógicos, útiles para controlar luces o velocidades de motores de forma suave.
5. **Puertos para Sensores Táctiles Capacitivos**: Son pines especiales que pueden detectar cuando un objeto conductor, como un dedo, toca la superficie del pin.
6. **Pines 1/0 del Procesador RTC de Bajo Consumo**: Son pines que se pueden usar cuando el ESP32 está en un modo de bajo consumo para mantener algunas funciones activas.
7. **Interfaz para Tarjetas SD**: Son pines que permiten conectar una tarjeta SD para almacenar y leer datos, como música o archivos.
8. **SPI (Interfaz Periférica en Serie)**: Es como una autopista de alta velocidad que permite que el ESP32 se comunique rápidamente con otros dispositivos, como sensores, pantallas y tarjetas de memoria. Es útil para enviar y recibir datos de manera rápida y eficiente entre el ESP32 y estos dispositivos.
9. **Bus I2C (Wire)**: Es un bus que permite que el ESP32 se comunique con otros dispositivos, como sensores, usando solo dos cables.

10. **Bus VSPI**: Es un bus de comunicación que se puede usar para conectar otros dispositivos que necesitan transmitir datos de manera rápida y sincronizada.
11. **Interfaces Seriales**: Son conexiones que permiten que el ESP32 se comunique con otros dispositivos en serie, como módulos GPS o impresoras.
12. **Bus HSPI**: Es otro bus de comunicación similar al VSPI, útil para conectar dispositivos que necesitan una alta velocidad de transferencia de datos.

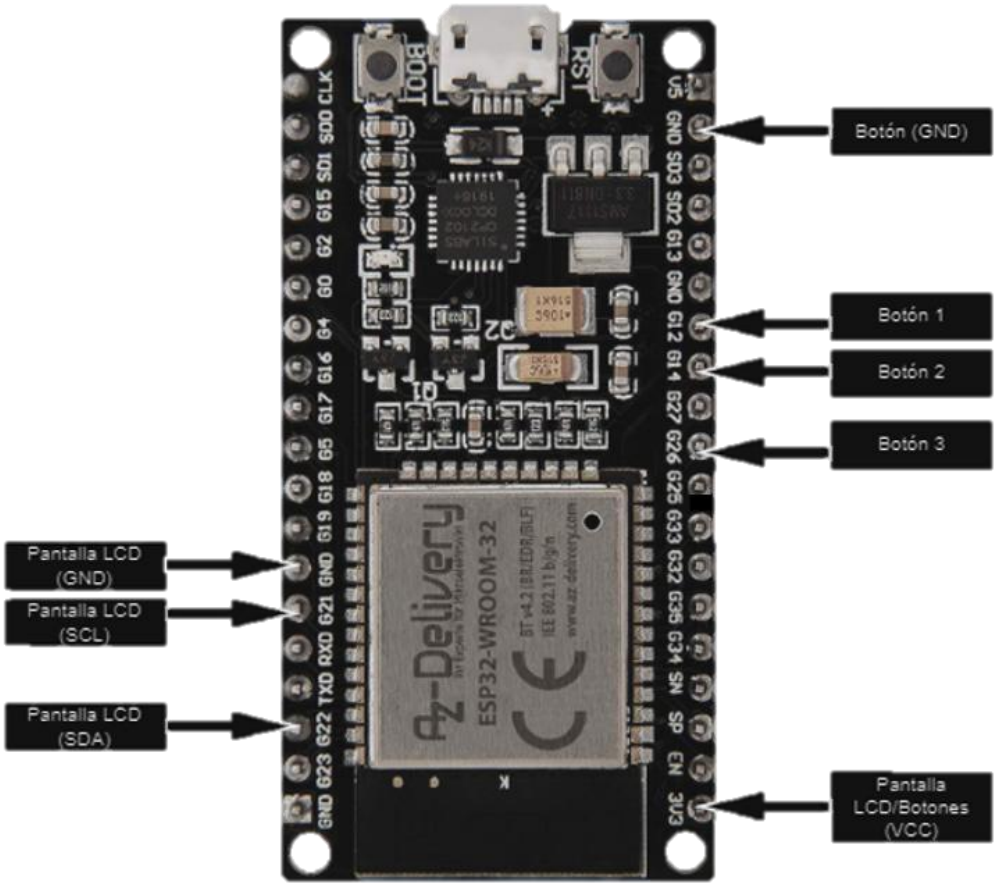
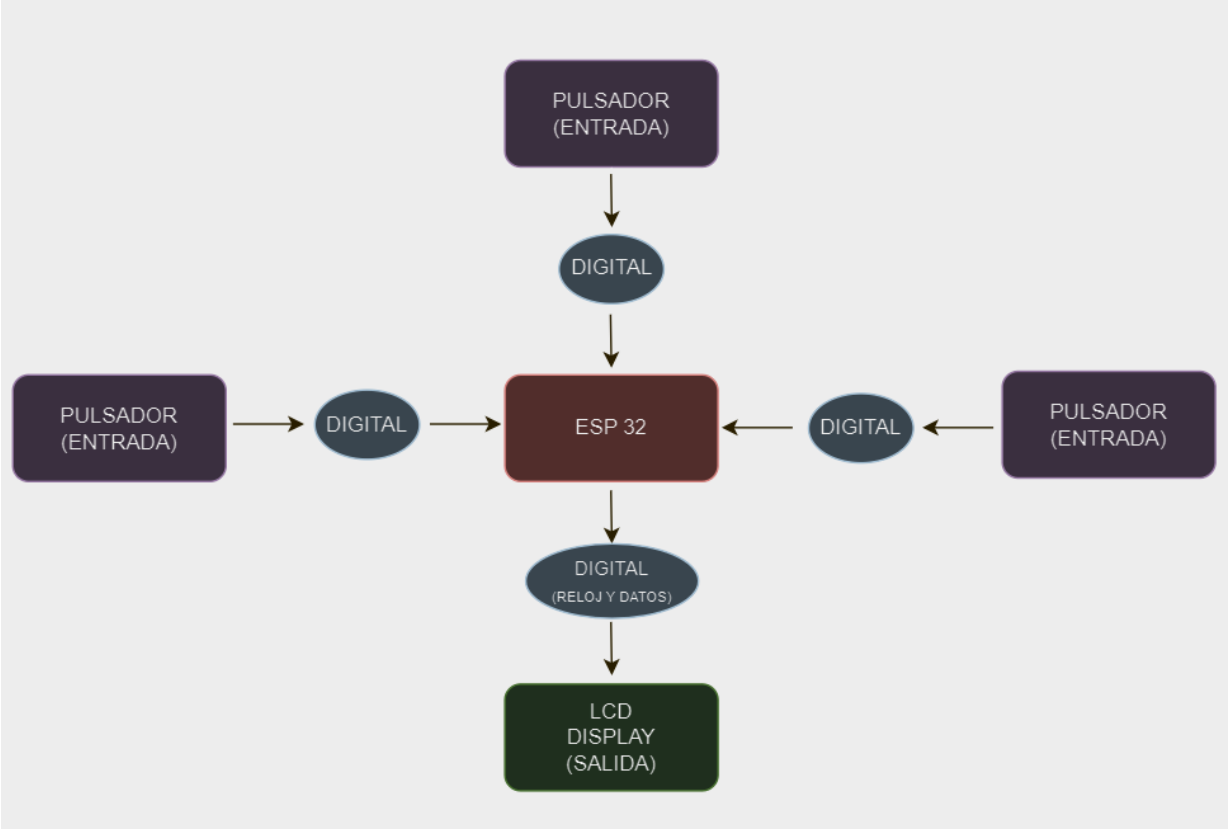
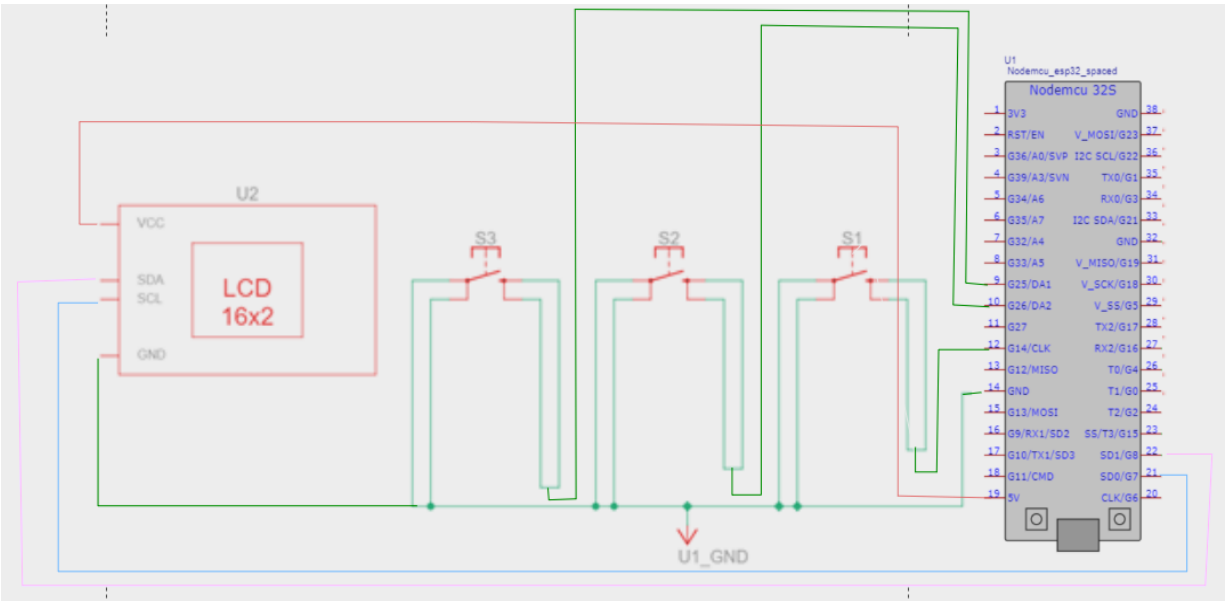


Diagrama de conexionado:

- SIMPLE



- COMPLETO



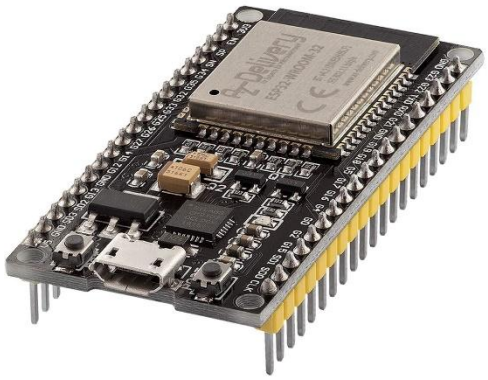
1. Conexión de Tierra (GND):
    - Esp32 a Display LCD: La conexión a tierra (GND) proporciona un camino de retorno para la corriente eléctrica y asegura un nivel de referencia común entre el ESP32 y el display LCD. Esto garantiza que las señales se puedan transmitir correctamente entre ambos dispositivos y evita problemas de ruido eléctrico.
    - Esp32 a Botones: Al igual que con el display LCD, conectar los botones a tierra asegura un nivel de referencia común para su funcionamiento. Esto es necesario para que el ESP32 pueda leer correctamente el estado de los botones.
2. Conexión de Alimentación (5V):
    - Esp32 a Display LCD: La conexión de alimentación proporciona la energía necesaria para que el display LCD funcione correctamente. En el ESP32, se utiliza el pin de 5 V para suministrar la tensión necesaria para el funcionamiento del display.
    - F.A a ESP32: Esta conexión dota al ESP 32 de energía, gracias a una fuente de alimentación de 5V conectada a cualquier toma de corriente. Se conectará al ESP 32 con un adaptador micro USB.

3. Conexión de Comunicación I2C (SCL/SDA):
    - Esp32 a Display LCD: La comunicación I2C (Inter-Integrated Circuit) se utiliza para la transmisión de datos entre el ESP32 y el display LCD. El pin SCL (Serial Clock) se utiliza para sincronizar la transmisión de datos, mientras que el pin SDA (Serial Data) se utiliza para enviar los datos en sí.
4. Conexiones de los Botones:
    - Los botones se conectan a pines GPIO (General Purpose Input/Output) disponibles en el ESP32. Estos pines se utilizan para leer el estado de los botones (presionado o no presionado). Cuando se presiona un botón, el pin correspondiente se conecta a tierra, lo que permite al ESP32 detectar este cambio de estado y responder en consecuencia.

Estas conexiones son fundamentales para que el dispositivo funcione correctamente. Al proporcionar una conexión de tierra y alimentación adecuada, así como una comunicación efectiva entre el ESP32, el display LCD y los botones, garantizamos que el dispositivo pueda monitorizar el estado de los ordenadores de la red y permitir su encendido remoto de manera eficiente.

1.2.2.Componentes Hardware

1. ESP32:



Para qué sirve: El ESP32 actúa como el cerebro del dispositivo, gestionando la conexión a la red WiFi y la comunicación con otros dispositivos, así como controlando las operaciones del display LCD y la lectura de los botones.

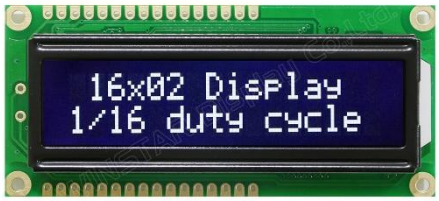
Especificaciones justificativas:

- Conectividad WiFi y Bluetooth integrada: Permite la conexión a la red WiFi para monitorizar los ordenadores y controlarlos remotamente.
- Potente CPU de doble núcleo: Proporciona la capacidad de procesamiento necesaria para gestionar múltiples tareas simultáneamente, como la comunicación con la red y la interacción con los componentes.
- Soporte para comunicación I2C, SPI y GPIO: Facilita la conexión con otros componentes, como el display LCD y los botones, a través de estos protocolos de comunicación.

Cómo se comunica con el resto de componentes: Se comunica con el display LCD a través del protocolo I2C y con los botones mediante pines GPIO configurados como entradas digitales.

Característica	Descripción
Voltaje de alimentación	5V
Voltaje de entrada / salida	3.3V
Corriente de Funcionamiento	min. 500mA
SoC	ESP32-WROOM 32
Frecuencia de Reloj	80MHz / 240MHz
RAM	512kB
Memoria Flash externa	4MB
Pines I / O	34
Interfaces	SPI, I2C, I2S, CAN, UART
Protocolos Wi-Fi	802.11 b/g/n (802.11n hasta 150 Mbps)
Frecuencia Wi-Fi	2.4 GHz - 2.5 GHz
Bluetooth	V4.2 - BLE y Bluetooth clásico
Antena inalámbrica	PCB
Dimensiones	56x28x13mm

2. Display LCD (pantalla):



Para qué sirve: El display LCD muestra información sobre el estado de los ordenadores en la red y las acciones realizadas, proporcionando una interfaz de usuario visual para interactuar con el dispositivo.

Especificaciones justificativas:

- Comunicación I2C: Permite una conexión sencilla y eficiente con el ESP32 utilizando solo dos cables, facilitando la integración en el dispositivo.
- Pantalla de tamaño adecuado y legible: Proporciona suficiente espacio para mostrar información relevante de manera clara y legible.
- Bajo consumo de energía: Minimiza el impacto en la duración de la batería o la fuente de alimentación del dispositivo.

Cómo se comunica con el resto de componentes: Se comunica con el ESP32 a través del protocolo I2C, utilizando los pines SCL y SDA para la transmisión de datos y comandos.

Característica	Descripción
Tamaño	16 caracteres x 2 líneas
Controlador	HD44780 compatible
Comunicación	Interfaz I2C (también conocida como interfaz serial I2C/TWI)
Voltaje de operación	5V (compatible con 3.3V)
Retroiluminación	Retroiluminación LED azul
Contraste ajustable	Potenciómetro para ajustar el contraste
Dirección I2C	Configurable mediante puente (0x20 por defecto)
Dimensiones	Aprox. 80mm x 36mm x 12mm

3. Botones:



Para qué sirve: Los botones permiten al usuario interactuar con el dispositivo, seleccionando opciones en el menú o enviando comandos para encender o apagar los ordenadores.

Especificaciones justificativas:

- Disponibilidad de pines GPIO en el ESP32: Permite conectar los botones directamente al ESP32 para su detección.
- Facilidad de uso y familiaridad para el usuario: Los botones son una interfaz de usuario intuitiva y ampliamente utilizada para la interacción con dispositivos electrónicos.

Cómo se comunica con el resto de componentes: Los botones se conectan a pines GPIO del ESP32 y se utilizan como entradas digitales para detectar el estado de los botones (presionado o no presionado).

4. Placa de Prototipado (Protoboard):



Para qué sirve: La placa de protoboard es una plataforma para construir circuitos electrónicos temporales. Permite conectar componentes electrónicos entre sí sin necesidad de soldadura, facilitando la creación y prueba de prototipos de circuitos.

Cómo se comunica con el resto de componentes: La placa de protoboard se comunica físicamente mediante cables que conectan los componentes y pines del Arduino entre sí.

5. Cables de Conexión:



Para qué sirve: Los cables de conexión se utilizan para establecer conexiones entre los componentes electrónicos y el esp32 o entre los componentes entre sí en la placa de protoboard.

Cómo se comunica con el resto de componentes: Los cables de conexión se conectan físicamente entre los pines del Arduino, los componentes electrónicos y la placa de protoboard para establecer las conexiones eléctricas necesarias.

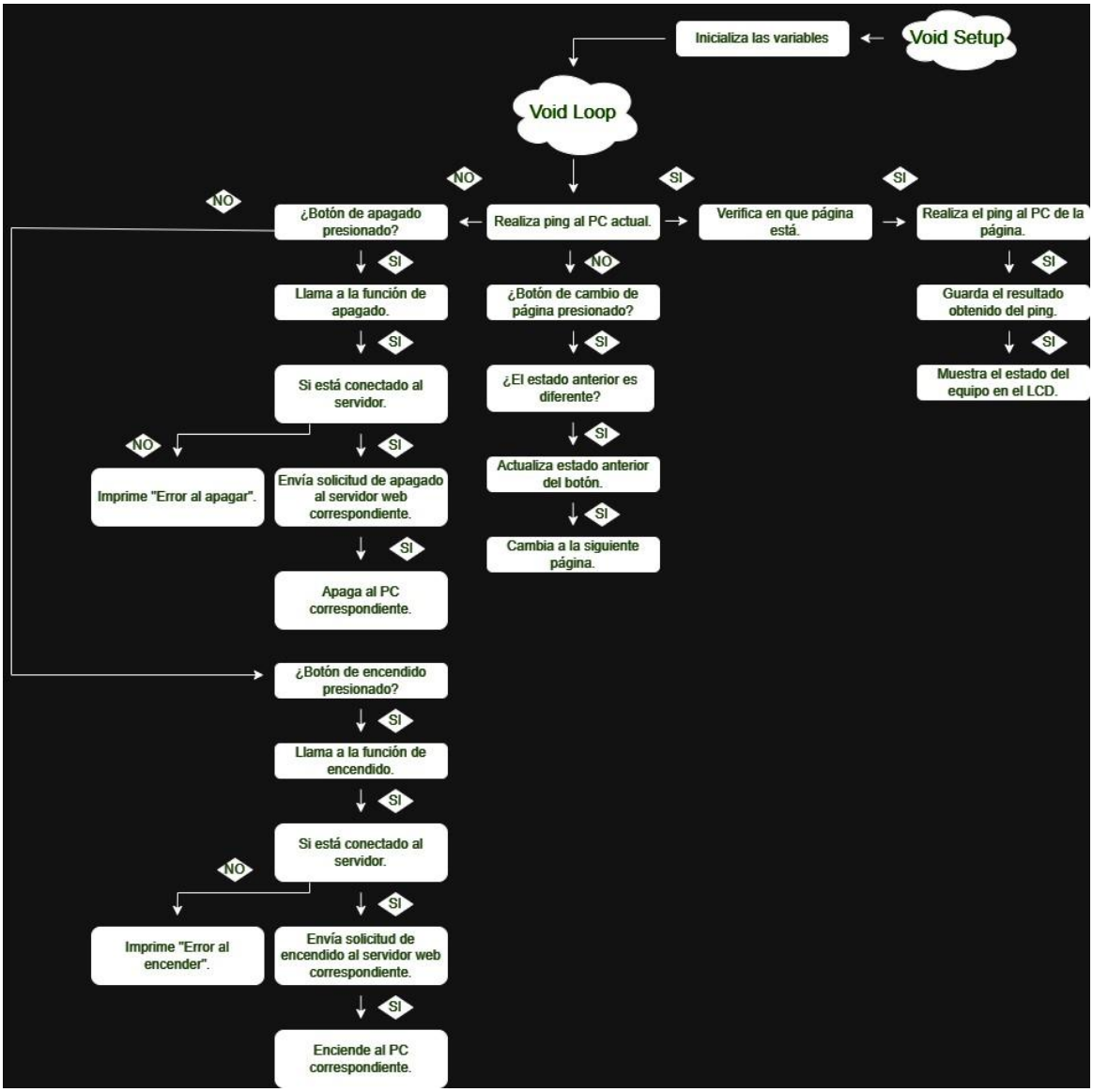
6. Fuente de alimentación:



Para qué sirve: es el sistema que suministra la energía eléctrica necesaria para su funcionamiento. Suministrará la energía al ESP 32.

Característica	Descripción
Entrada	100 a 240 V a 50/60 Hz de corriente alterna (AC).
Salida	corriente continua (DC) de 5V y 2.4 A máximo.
Conector	Micro USB.

1.2.3. Desarrollo software.



La librería <ESP32Ping.h> se descarga desde Github de la siguiente forma:

1. Ve al repositorio de la biblioteca en GitHub y haz clic en el botón verde "Code" y luego en "Download ZIP".

2. Extrae el archivo ZIP descargado en tu computadora.

3. Abre el IDE de Arduino, ve a "Sketch" en la barra de menú y selecciona "Include Library" -> "Add .ZIP Library...".

4. Navega hasta la ubicación donde extrajiste el archivo ZIP de la biblioteca y selecciónalo.

5. Arduino importará la biblioteca y la agregará a tu entorno de desarrollo.

1.3.Validación y conclusiones.

En el desarrollo del proyecto de control remoto de encendido y monitoreo de ordenadores mediante un ESP32, se identificaron varios puntos críticos que causaron errores. A continuación, se detallan los principales errores encontrados y sus respectivas soluciones:

Error al no poner la dirección correcta del LCD:

- Problema: El LCD se encendía, pero no mostraba nada por el display
- Solución: La dirección I2C del módulo LCD no se configuró correctamente en el código, el ESP32 no podía comunicarse con la pantalla, estaba utilizando una 0X20 en vez de 0X27.

Error al no utilizar librerías adecuadas de PING:

- Problema: Al compilar el código me generaba un error en la librería <WiFiPing.h>.
- Solución: Utilizar una librería de PING compatible con el ESP32, como la librería <ESP32Ping.h>. El uso de librerías inapropiadas o incompatibles para realizar el ping a las direcciones IP puede causar que el ESP32 no pueda verificar correctamente el estado de los equipos.

Error al no encender el ordenador debido a que ciertos ordenadores en suspensión desactivan el adaptador de red:

- Problema: Algunos ordenadores desactivan el adaptador de red cuando entran en modo de suspensión, lo que impide que puedan recibir señales de encendido remoto.
- Solución: En algunos casos se podría configurar el adaptador de red en el administrador de dispositivos para que permanezca activo en modo de suspensión.

Para mejorar el proyecto se podría agregar monitoreo del rendimiento de la CPU y registrar la última hora de apagado:

- Utilizar TakeScheduler.h para medir la carga de la CPU y visualizarla en tiempo real.
- Registrar la última hora de apagado utilizando RTClib.h.
- Mostrar esta información en el LCD o a través de la conexión web.
- Estas adiciones enriquecerán el proyecto y demostrarán comprensión avanzada de monitoreo y manejo de datos.

En cuanto a configuración de red solo es necesario configurar las IPs de los ordenadores como estáticas:

- Accede a la configuración de red del PC (en Windows: Panel de Control > Centro de redes y recursos compartidos > Cambiar configuración del adaptador).
- Selecciona la conexión de red adecuada (Ethernet o Wi-Fi) y haz clic en Propiedades.
- Selecciona Protocolo de Internet versión 4 (TCP/IPv4) y haz clic en Propiedades nuevamente.
- Ingresa la dirección IP estática, la máscara de subred y la puerta de enlace predeterminada según la configuración de la red local, estando dentro del rango de la LAN.

Configurar Servidor Web:

Para la crear y lanzar el servidor web opté por usar Flask. Esta herramienta de Python es muy útil para crear aplicaciones web de manera sencilla y rápida.

Para ello se crea en un documento de texto el contenido del servidor y se guarda con el formato ".py", luego en el cmd se instala Flask, se cambia el directorio del archivo que guardamos con él ".py" y por ultimo ejecutamos el flask. Así tendríamos corriendo el servidor.

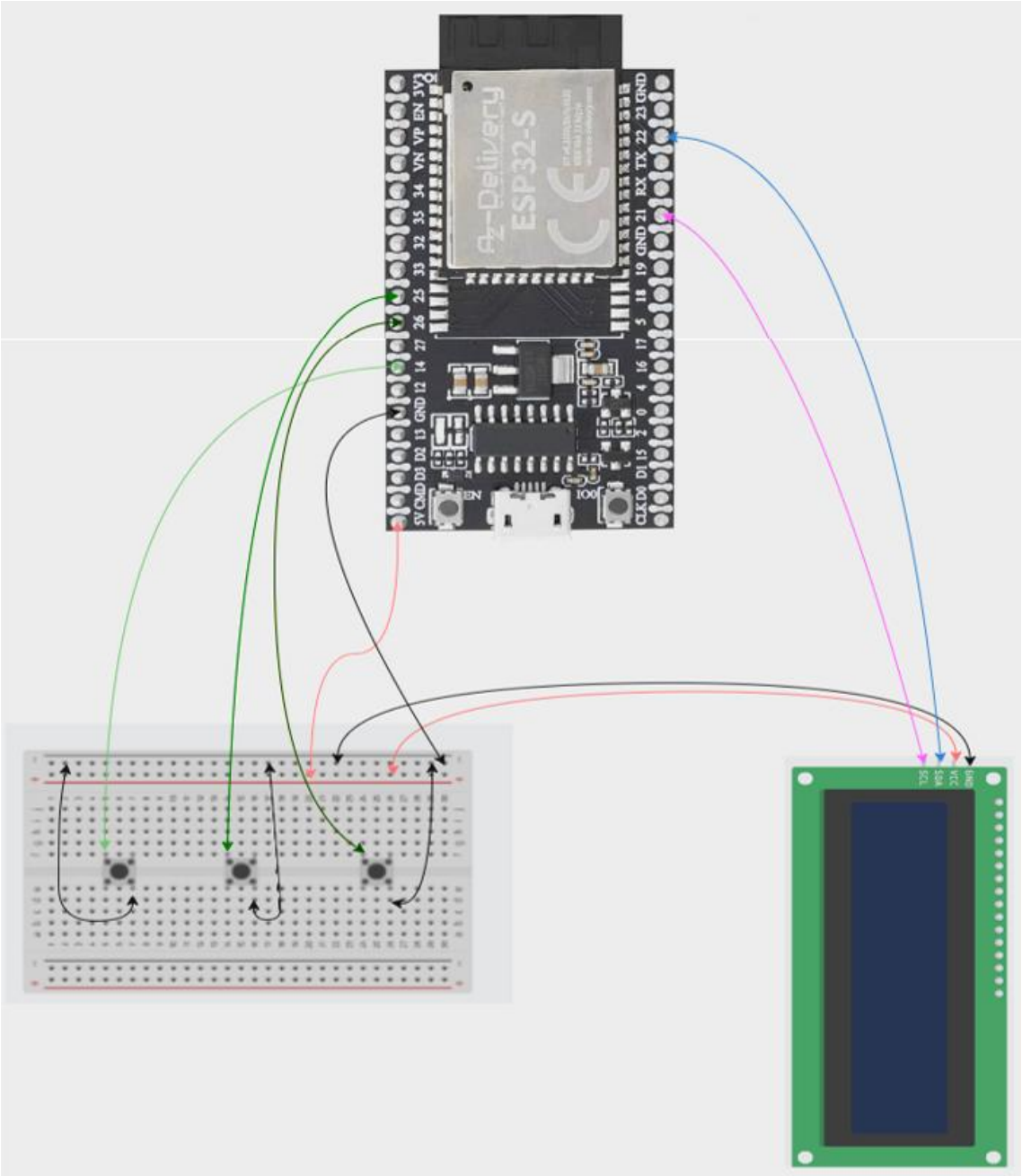
2. Presupuesto.

Componentes	Precio	Cantidad	Total	Enlace
Esp 32	11,99 €	1	11,99 €	https://amzn.eu/d/8SYoCQS
Pulsador	5,99 €	100	5,99 €	https://amzn.eu/d/5GeH4jo
LCD	8,95 €	1	8,95 €	https://amzn.eu/d/4VYJBTu
Placa protoboard	5,90 €	1	5,90 €	https://amzn.eu/d/2JlucVD
Cables	9,59 €	120	9,59 €	https://amzn.eu/d/2urKbtl
F.A.	17,71€	1	17,71 €	https://amzn.eu/d/7vnZxwu
			60,13 €	

Mano de obra	Precio	Cantidad	Total
Implementación servidor web, interfaz y código	19 € / h	12 h	228 €
Montaje	19 € / h	30 min	9,5 €
Pruebas	19 € / h	30 min	9,5 €
Total 307, 13€			247 €

3. Planos.

3.1. Montaje eléctrico.



4. Anexos.

4.1. Código fuente.

Arduino:

```
#include <WiFi.h>  // Biblioteca que permite la utilización de funciones y objetos mediante el manejo de conexiones Wi-Fi en el ESP32.
#include <LiquidCrystal_I2C.h> // Biblioteca que simplifica la comunicación con pantallas LCD mediante el protocolo I2C.
#include <ESP32Ping.h> // Biblioteca que facilita la realización de pings a dispositivos en una red Wi-Fi utilizando el ESP32.
#include <ESPAsyncWebServer.h> // Permite crear un servidor web asíncrono en el ESP32.

#define LCD_ADDRESS 0x3F // Especifica la dirección I2C de la pantalla LCD.
#define LCD_ROWS 2 // Define el número de filas en la pantalla LCD.
#define LCD_COLS 16 // Define el número de columnas en la pantalla LCD.

#define BUTTON_PIN 14 // Asigna el pin utilizado para el botón de cambio de página.
#define BUTTON_PIN_APAGAR 25 // Asigna el pin utilizado para el botón de apagar.
#define BUTTON_PIN_ENCENDER 26 // Asigna el pin utilizado para el botón de encender.

LiquidCrystal_I2C lcd(LCD_ADDRESS, LCD_COLS, LCD_ROWS); // Esto declara un objeto para controlar una pantalla LCD con comunicación I2C.

// Credenciales de la red.
const char* ssid = "Galaxy A51 4F42";
const char* password = "galaxy51a";


// Dirección IP y puerto del servidor web del PC 1.
const IPAddress serverIP1(192, 168, 238, 157);
const int serverPort1 = 5000;  // Puerto donde está ejecutando Flask

// Dirección IP y puerto del servidor web del PC 2
const IPAddress serverIP2(192, 168, 238, 158);
const int serverPort2 = 5000;  // Puerto donde está ejecutando Flask

// Dirección IP de las PCs.
IPAddress ip_pc1(192, 168, 238, 157);
IPAddress ip_pc2(192, 168, 238, 158);

int currentPage = 1; // Define una variable entera llamada PáginaActual y la inicializa con el valor 1.
bool buttonState = false; // Declara una variable booleana llamada EstadoBotón y la inicializa como falsa.
bool previousButtonState = false; // Declara una variable booleana llamada EstadoPrevioBotón y la inicializa como falsa.

AsyncWebServer server(80); // Crea un objeto server del tipo AsyncWebServer que escucha en el puerto 80 para manejar solicitudes web asincronas.
```

```
void setup() {
  Serial.begin(115200); // Inicia la comunicación serial con una velocidad de baudios de 115200.


  lcd.init(); // Inicializar LCD.
  lcd.backlight(); // Enciende la retroiluminación de la pantalla LCD.
  lcd.setCursor(0, 0); // Establece el cursor en la posición (0, 0) de la pantalla LCD.
  lcd.print("Estado del equipo"); // Imprime el texto "Estado del equipo" en la pantalla LCD.
  Serial.println("LCD inicializado"); // Imprime "LCD inicializado" a través de la comunicación serial.

  // Conectar a WiFi
  Serial.println("Conectando a WiFi..."); // Imprime "Conectando a WiFi..." a través de la comunicación serial.
  WiFi.begin(ssid, password); // Inicia el proceso de conexión a una red Wi-Fi utilizando el nombre de red y la contraseña proporcionados.
  while (WiFi.status() != WL_CONNECTED) { //Espera a que este conectado a la WiFi
    delay(1000);
    Serial.println("Conectando a WiFi..."); // Imprime en el puerto serie repetidamente "Conectando a WiFi..."
    lcd.setCursor(0, 1);
    lcd.print("Conectando..."); // Imprime  "Conectando..." en la pantalla LCD.
  }

  Serial.println("Conexión WiFi establecida"); // Imprime "Conexión WiFi establecida" a través de la comunicación serial.
  lcd.setCursor(0, 1); // Establece el cursor en la segunda fila de la pantalla LCD.
  lcd.print("WiFi conectado"); //  Imprime "WiFi conectado" en la pantalla LCD.
  delay(2000); // Pequeño retraso para mostrar el mensaje de conexión WiFi
  lcd.clear(); // Borra el contenido de la pantalla LCD.
  Serial.println("Conexión WiFi establecida y mensaje LCD actualizado"); //  Imprime "Conexión WiFi establecida y mensaje LCD actualizado" a través de la comunicación serial.

  pinMode(BUTTON_PIN, INPUT_PULLUP); // Configura el pin BUTTON_PIN como entrada con resistencia de pull-up interna.
  pinMode(BUTTON_PIN_APAGAR, INPUT_PULLUP);// Configura el pin BUTTON_PIN_APAGAR como entrada con resistencia de pull-up interna.
  pinMode(BUTTON_PIN_ENCENDER, INPUT_PULLUP);// Configura el pin BUTTON_PIN_ENCENDER como entrada con resistencia de pull-up interna.

  // Iniciar servidor web en el ESP32
  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){  // Configura un manejador para la ruta raíz que responde a solicitudes GET con un mensaje.
    request->send(200, "text/plain", "Presiona el botón 25 para apagar el equipo\nPresiona el botón 26 para encenderlo");
  });

  server.begin(); // Inicia el servidor web.
  Serial.println("Servidor web iniciado en ESP32"); // Imprime un mensaje indicando que el servidor web se ha iniciado.
}
```

```
void loop() {
  Serial.println("Intentando ping..."); //  Esta linea imprime "Intentando ping..." en el monitor serial.
  bool pingResult; //  Aquí se declara una variable booleana, que se utilizará para almacenar el resultado del ping.

  if (currentPage == 1) { // Se realiza un ping al PC correspondiente según el valor de currentPage (PC1 si currentPage es 1, PC2 en otro caso), almacenando el resultado en pingResult.
    pingResult = Ping.ping(ip_pc1);
  } else {
    pingResult = Ping.ping(ip_pc2);
  }

  lcd.setCursor(0, 0); // Aquí se establece el cursor en la primera fila (fila 0) y primera columna (columna 0) de la pantalla LCD.
  lcd.print("Estado del equipo"); // Imprime por el LCD "Estado del equipo".

  if (currentPage == 1) { // Imprime por el LCD el número de PC según el valor de currentPage (PC1 si currentPage es 1, PC2 en otro caso).
    lcd.setCursor(0, 1);// Ajusta el cursor en la segunda fila.
    lcd.print("PC 1: ");// Imprime "PC 1: ".
  } else {//De lo contrario.
    lcd.setCursor(0, 1);// Ajusta el cursor en la segunda fila.
    lcd.print("PC 2: ");// Imprime "PC 2: ".
  }

  Serial.print("Ping a PC "); // Imprime en el monitor serial "Ping a PC".
  Serial.print(currentPage); // Imprime el valor de currentPage.
  Serial.print(": ");  // Imprime ":".
  Serial.println(pingResult ? "exitoso" : "fallido"); // Imprime "exitoso" si pingResult es verdadero, y "fallido" si pingResult es falso, en el monitor serial.

  lcd.setCursor(5, 1); //Establece el cursor en la quinta columna de la segunda fila de la pantalla LCD.
  if (pingResult) {
    lcd.print("on ");
  } else {
    lcd.print("off");
  }

  // Verificar el estado del botón de cambio de página
  buttonState = digitalRead(BUTTON_PIN); //  Lee el estado del botón conectado al pin definido como BUTTON_PIN y almacena este estado en la variable buttonState.
  if (buttonState != previousButtonState && buttonState == LOW) { // Cambia la página de 1 a 2 cuando se presiona el botón y de 2 a 1 cuando se presiona nuevamente.
    currentPage = currentPage == 1 ? 2 : 1; // Cambiar la página
    delay(1000); // Pequeño retraso para evitar rebotes
  }
```

```
  .
  previousButtonState = buttonState; //  Se actualiza el valor de  previousButtonState  con el estado actual del botón buttonState, para que esté listo para la próxima iteración del bucle.

   // Leer el estado de los botones de encendido y apagado. Y guarda su estado en buttonStateApagar/Encender.
  int buttonStateApagar = digitalRead(BUTTON_PIN_APAGAR);
  int buttonStateEncender = digitalRead(BUTTON_PIN_ENCENDER);

  // Si el botón de apagado está presionado, enviar la solicitud de apagado al servidor web del equipo correspondiente
  if (buttonStateApagar == LOW) {//  Si el botón de apagado está presionado.
    Serial.println("Botón de apagado presionado");// Imprime  "Botón de apagado presionado" en monitor serial.
    lcd.setCursor(0, 1);// Ajusta el cursor en la fila 1.
    lcd.print("Apagando...      ");// Imprime  "Apagando..." en el LCD.
    enviarSolicitudApagado();// Envía la solicitud de apagado al servidor web del equipo correspondiente.
    delay(1000);  // Pausa para evitar múltiples pulsaciones del botón
    lcd.clear();  // Limpiar la línea después de un breve retraso
  }

  // Si el botón de encendido está presionado, enviar la solicitud de encendido al servidor web del equipo correspondiente.
  if (buttonStateEncender == LOW) {// Si el botón de encendido está presionado.
    Serial.println("Botón de encendido presionado"); // Imprime  "Botón de encendido presionado" en monitor serial.
    lcd.setCursor(0, 1);// Ajusta el cursor en la fila 1.
    lcd.print("Encendiendo...  ");//Imprime  "Encendiendo..." en el LCD.
    enviarSolicitudEncendido();// Envía la solicitud de encendido al servidor web del equipo correspondiente.
    delay(1000);  // Pausa para evitar múltiples pulsaciones del botón
    lcd.clear();  // Limpiar la línea después de un breve retraso
  }

  delay(1000);
}
```

```
void enviarSolicitudApagado() {
  // Realizar una solicitud GET al servidor web del equipo correspondiente para apagarlo
  WiFiClient client;//  Se declara un objeto que se utilizará para establecer la conexión con el servidor.
  IPAddress serverIP = currentPage == 1 ? serverIP1 : serverIP2; // Asigna a serverIP serverIP1 si currentPage es 1, de lo contrario, asigna serverIP2
  int serverPort = currentPage == 1 ? serverPort1 : serverPort2;  // Asigna a serverPort serverPort1 si currentPage es 1, de lo contrario, asigna serverPort2

  if (client.connect(serverIP, serverPort)) {//  Se intenta establecer una conexión con el servidor utilizando la dirección IP y el puerto obtenidos anteriormente.
    Serial.println("Conectado al servidor, enviando solicitud de apagado..."); //  Se imprime un mensaje indicando que se ha establecido la conexión con el servidor y que se enviará una solicitud de apagado.
    client.print("GET /shutdown HTTP/1.1\r\n"); //Esta línea envía una solicitud GET al servidor para apagarlo. "GET" indica el método HTTP utilizado, "/shutdown" es la ruta del recurso al que se accede en el servidor, y
"HTTP/1.1" es la versión del protocolo HTTP utilizada. \r\n representa un retorno de carro  y un salto de línea, que son caracteres especiales para indicar el final de una línea en HTTP.
    client.print("Host: ");  //Esta línea especifica el host al que se está accediendo. En este caso, imprime "Host: ".
    client.print(serverIP.toString()); //  Aquí se imprime la dirección IP del servidor. Convierte la dirección IP en una cadena de texto para poder enviarla al servidor.
    client.print("\r\n"); //Indica el final de la línea en el protocolo HTTP.
    client.print("Connection: close\r\n\r\n"); // Después de enviar esta línea, se añaden dos \r\n adicionales para indicar el final de la solicitud HTTP.
    client.flush(); // Se envían los datos.
    client.stop(); // Se  detiene la conexión con el servidor.
    Serial.println("Solicitud de apagado enviada"); // Se imprime un mensaje indicando que se ha enviado la solicitud de apagado con éxito.
    lcd.setCursor(0, 1);//Ajusta el cursor en la fila 1.
    lcd.print("Apagado enviado "); // Imprime en el LCD "Apagado enviado"
    delay(2000);  // Mostrar mensaje durante 2 segundos
    lcd.clear();  // Limpiar la pantalla después del mensaje
  } else {
    Serial.println("Error al conectar con el servidor web del equipo"); // Imprime en el monitor serial  "Error al conectar con el servidor web del equipo".
    lcd.setCursor(0, 1); // Ajusta el cursor en la fila 1.
    lcd.print("Error al apagar "); // Imprime en el LCD  "Error al apagar ".
    delay(2000);  // Mostrar mensaje durante 2 segundos
    lcd.clear();  // Limpiar la pantalla después del mensaje
  }
}
```

```
void enviarSolicitudEncendido() {
  // Realizar una solicitud GET al servidor web del equipo correspondiente para encenderlo
  WiFiClient client;//  Se declara un objeto que se utilizará para establecer la conexión con el servidor.
  IPAddress serverIP = currentPage == 1 ? serverIP1 : serverIP2; // Asigna a serverIP serverIP1 si currentPage es 1, de lo contrario, asigna serverIP2
  int serverPort = currentPage == 1 ? serverPort1 : serverPort2; // Asigna a serverPort serverPort1 si currentPage es 1, de lo contrario, asigna serverPort2

  if (client.connect(serverIP, serverPort)) {// Intenta establecer una conexión con el servidor.
    Serial.println("Conectado al servidor, enviando solicitud de encendido...");// Imprime un mensaje en el monitor serial indicando que la conexión al servidor ha sido establecida y que se está enviando la solicitud de
encendido.
    client.print("GET /startup HTTP/1.1\r\n");// Envía una solicitud GET al servidor para iniciar el servidor. Se especifica la ruta del recurso /startup y la versión del protocolo HTTP.
    client.print("Host: ");// Imprime el encabezado "Host".
    client.print(serverIP.toString());// La dirección IP del servidor se convierte en una cadena de texto utilizando " serverIP.toString  "
    client.print("\r\n");// Se envía al servidor.
    client.print("Connection: close\r\n\r\n"); // Especifica que la conexión se cerrará después de completar la solicitud.
    client.flush();// Envía la información al servidor.
    client.stop(); // Cierra la sesión con el sevidor.
    Serial.println("Solicitud de encendido enviada");//  Imprime un mensaje en el monitor serial indicando que la solicitud de encendido ha sido enviada con éxito.
    lcd.setCursor(0, 1);// Ajusta el cursor en la fila 1.
    lcd.print("Encendido enviado");// Imprime "Encendido enviado" por el LCD.
    delay(2000);  // Mostrar mensaje durante 2 segundos
    lcd.clear();  // Limpiar la pantalla después del mensaje
  } else {
    Serial.println("Error al conectar con el servidor web del equipo");// Imprime por el monitor serial.
    lcd.setCursor(0, 1);// Ajusta el cursor en la fila 1.
    lcd.print("Error al encender"); // Imprime por el LCD.
    delay(2000);  // Mostrar mensaje durante 2 segundos
    lcd.clear();  // Limpiar la pantalla después del mensaj
  }
}
```

Flask:

- Server 1.

```
from flask import Flask                 //Importa Flask para crear la aplicación web.

import os               //Ejecuta comandos del sistema operativo.

app = Flask(__name__)               //Crea la aplicación web, sería “app”.

@app.route('/')                 // Define una ruta para la URL raíz (/). Llamará a home.

def home():             //Esta función maneja las solicitudes a la URL raíz.

    return "Servidor Flask en funcionamiento."              // Muestra esta cadena de texto cuando se acceda a la URL raíz.

@app.route('/shutdown', methods=['GET'])                // Define una ruta para la URL /shutdown. Esta ruta solo acepta solicitudes GET.

def shutdown():                 //Define una función llamada shutdown que maneja las solicitudes a la URL /shutdown.

    os.system("shutdown /s /t 0")               //Utiliza el módulo os para ejecutar el comando de apagado de Windows. Este comando apaga el pc inmediatamente.

    return "PC apagada."// Devuelve una cadena de texto que se mostrará en el navegador web confirmando que el PC se ha apagado.

@app.route('/startup', methods=['GET'])         //Define una ruta para la URL /startup. Esta ruta solo acepta solicitudes GET.

def startup():          //Define una función llamada startup que maneja las solicitudes a la URL /startup.

    os.system("start") // Utiliza el módulo os para ejecutar el comando start de Windows.

    return "PC encendida."                  // Devuelve una cadena de texto que se mostrará en el navegador web confirmando que la PC se ha "encendido".

if __name__ == '__main__':                  // Esta línea verifica si el archivo se está ejecutando directamente.

    app.run(host='192.168.238.157', port=5000)                  //Esta línea inicia el servidor web Flask.
```

- Server 2.

```
from flask import Flask
import os


app = Flask(__name__)
@app.route('/')
def home():
    return "Servidor Flask en funcionamiento."
@app.route('/shutdown', methods=['GET'])
def shutdown():
    os.system("shutdown /s /t 0")
    return "PC apagada."
@app.route('/startup', methods=['GET'])
def startup():
    os.system("start")
    return "PC encendida."
if __name__ == '__main__':
    app.run(host='192.168.238.158', port=5000)
```

4.2.Manuales.

4.2.1.Manual de uso.

Introducción

El sistema de monitoreo y control remoto desarrollado con ESP32 permite supervisar y gestionar el estado de dispositivos remotos, como computadoras, a través de una conexión Wi-Fi. Este manual proporciona instrucciones detalladas sobre cómo configurar y utilizar el sistema de manera eficiente.

Requisitos Previos

- Dispositivo ESP32 con firmware previamente cargado.
- Acceso a una red Wi-Fi funcional.

Configuración Inicial

- Conecte el ESP32 a una fuente de alimentación adecuada.
- Asegúrese de que el ESP32 esté conectado a la misma red Wi-Fi que los dispositivos remotos.
- Encienda el ESP32 y permita que se inicie completamente.

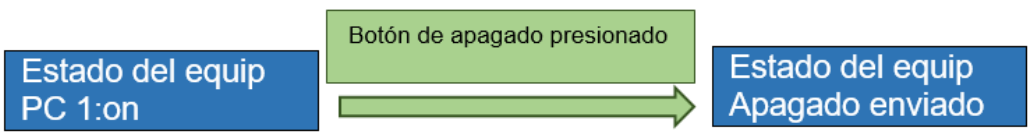
Interfaz de Usuario

El sistema utiliza una pantalla LCD y botones físicos para proporcionar una interfaz intuitiva.

Pantalla LCD:

- Muestra el estado del equipo y la conexión Wi-Fi.
- La primera línea muestra: “Estado del equipo”.
- La segunda línea muestra: estado de la conexión Wi-Fi.

Ilustración de menú en el LCD:




Estado del equip
PC 1:on


Estado del equip
PC 2:on


Estado del equip
Apagado enviado


Estado del equip
Encendido enviad

Botones:

- Botón 14: Cambia entre los dispositivos remotos (PC 1 y PC 2).
- Botones 25 y 26: Apagan y encienden respectivamente el PC seleccionado.

Uso del Sistema

1. Encienda el sistema y espere a que se conecte a la red Wi-Fi.
2. Utilice el botón 14 para seleccionar el dispositivo remoto deseado (PC 1 o PC2).
3. Monitoree el estado del dispositivo remoto en la pantalla LCD.
4. Para apagar o encender el dispositivo remoto seleccionado:
5. Presione el botón 25 para apagar o el botón 26 para encender.
6. Espere a que se complete la solicitud y observe la confirmación en el display.

Consideraciones Adicionales

- Verifique que las direcciones IP de los dispositivos remotos y del servidor web sean correctas y accesibles desde la red Wi-Fi.
- Asegúrese de tener una conexión Wi-Fi estable para el correcto funcionamiento del sistema.

Este manual proporciona una guía clara y concisa para configurar y utilizar el sistema de monitoreo y control remoto.

4.2.2.Manual de mantenimiento.

El mantenimiento regular es esencial para garantizar el funcionamiento continuo y confiable del sistema de monitoreo y control remoto basado en ESP32. Este manual proporciona una guía detallada sobre cómo realizar chequeos de mantenimiento, identificar posibles averías y aplicar soluciones adecuadas para mantener el sistema en óptimas condiciones de funcionamiento.

Chequeos de Mantenimiento Recomendados

Verificación de Conexión Wi-Fi:

- Asegúrese de que el ESP32 esté conectado a la red Wi-Fi verificando el display, este mensaje "Conectando a WiFi..." no debería permanecer por más de 20 segundos.
- Revise regularmente la intensidad de la señal Wi-Fi para garantizar una conexión estable.
- Revise las luces del router o punto de acceso.

- El ESP32 debe conectarse a un punto de acceso WiFi (AP) que opere en la misma red IP que las PCs que se desean controlar.
- Es importante tener en cuenta que podrían existir reglas que bloqueen determinadas direcciones MAC, por lo tanto, será necesario incluir la dirección MAC del ESP32 en la lista de dispositivos permitidos para evitar bloqueos.
- Si el ESP32 y las PCs se encuentran en redes IP diferentes, asegurarse de que se configuren correctamente las rutas de enrutamiento necesarias entre ambas redes. Además, verifica que los routers involucrados no tengan reglas de bloqueo que puedan impedir la comunicación entre el ESP32 y las PCs.

Inspección Visual:

Examine visualmente todos los componentes del sistema, incluyendo el ESP32, la pantalla LCD, botones físicos y dispositivos de red, en busca de daños físicos, cables sueltos o conexiones defectuosas.

Prueba de Funcionamiento:

- Realice pruebas periódicas para verificar el funcionamiento adecuado de la pantalla LCD y los botones físicos.
- Monitoree y controle dispositivos remotos para confirmar la comunicación bidireccional entre el ESP32 y los dispositivos.

Posibles Averías y Soluciones

Problemas de Conexión Wi-Fi:

- Avería: El ESP32 no se conecta a la red Wi-Fi.
- Solución: Verifique las credenciales de conexión Wi-Fi y reinicie el ESP32. Si el problema persiste, verifique la disponibilidad de la red Wi-Fi y la configuración del enrutador.

Falla en la Pantalla LCD:

- Avería: La pantalla LCD muestra caracteres incorrectos o no muestra información.
- Solución: Revise las conexiones entre el ESP32 y la pantalla LCD. Reemplace la pantalla si es necesario. Actualizar el firmware del ESP32 si el problema persiste.

Botones Físicos No Responden:

- Avería: Los botones físicos no funcionan cuando se presionan.
- Solución: Verifique las conexiones de los botones y reemplace los botones defectuosos. Limpie los contactos de los botones si es necesario.

Problemas de Comunicación con Dispositivos Remotos:

- Avería: El sistema no puede comunicarse con los dispositivos remotos.
- Solución: Verifique las direcciones IP de los dispositivos remotos y la configuración de red.

Programación de Mantenimiento Preventivo

- Realice chequeos de mantenimiento completos cada mes para detectar y corregir posibles problemas antes de que afecten el funcionamiento del sistema.
- Mantenga un registro de mantenimiento que incluya fechas de inspección, problemas detectados y acciones correctivas tomadas.

Este manual proporciona una guía práctica para el mantenimiento efectivo del sistema de monitoreo y control remoto. Al seguir estos pasos, puede garantizar la fiabilidad y el rendimiento óptimo del sistema a lo largo del tiempo.

5.Bibliografía.

Github. Librería ESP32Ping.

https://github.com/marian-craciunescu/ESP32Ping

YouTube. La pantalla LCD I2C no muestra texto.

https://youtu.be/eePEHdewf1o?si=PCXcmp64LKtKoBhB

YouTube. Como instalar Python y Flask.

https://youtu.be/uxZuFm5tmhM?si=BhH-D6kzliBBqsgV

Arduino Forum. Como usar ESP32Ping.

https://forum.arduino.cc/t/how-to-ping-remote-host-using-esp32-server/645607

Arduino.CC .Uso de librería WiFi.

https://www.arduino.cc/reference/en/libraries/wifi/

AZDelivery. Información y características sobre el ESP32.

https://www.az-delivery.de/en/products/esp32-developmentboard?variant=6119648460827

YouTube. Explicación de los pines del ESP 32.

https://youtu.be/BXi50c89YZM?si=F0N6YAYDSJjZZm-c

Windward Software. Deshabilitar el modo suspensión de la tarjeta de red.

https://support.windwardsoftware.com/support/solutions/articles/66000487622-disable-power-saving-and-power-options-in-windows

YouTube. “No such file directory”.

https://youtu.be/igdBT0hwsJc?si=djybzoxukKIG3Omq

Minibots.Wordpress. ESP32 o Raspberry.

https://minibots.wordpress.com/2021/04/06/pruebas-de-rendimiento-de-raspberry-pi-pico-y-comparativa-con-plataformas-arduino-esp8266-y-esp32/#:~:text=El%20nuevo%20procesador%20de%20Raspberry,incluye%20conectividad%20wifi%20y%20Bluetooth.

RBTechies. ESP 32 o Arduino.

https://rbtechies.com.br/es-arduino-vs-esp32-cual-es-la-mejor-opcion-para-tu-proyecto/

Arduino Forum. Hacer ping desde ESP 32.

https://forum.arduino.cc/t/how-to-ping-remote-host-using-esp32-server/645607/10

Reddit. Apagar pc desde ESP 32.

https://www.reddit.com/r/esp32/comments/onek5c/is_it_be_possible_to_shutdown_a_pc_using_an_esp32/